

P2435R0

2021 Summer Library Evolution Poll Outcomes

Published Proposal, 2021-09-09

Author:

[Bryce Adelstein Leibach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: [P2138R4] Rules of Design <=> Wording Engagement
- 3.2 Poll 2: [P2372R1] Fixing Locale Handling In Chrono Formatters
- 3.3 Poll 3: [P1206R6] `ranges::to`
- 3.4 Poll 4: [P0533R8] `constexpr` For `<cmath>` And `<cstdlib>`
- 3.5 Poll 5: [P2273R2] Making `unique_ptr` `constexpr`

References

Informative References

§ 1. Introduction

In Summer 2021, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2333R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 36 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Make a joint Library Evolution and Evolution plenary to adopt [P2138R4] (Rules of Design <=> Wording Engagement) as the official process of the C++ Evolution groups.	5	14	2	6	6	No consensus.
Poll 2: Send [P2372R1] (Fixing Locale Handling In Chrono Formatters) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item), with the recommendation that implementations retroactively apply it to C++20.	21	10	0	0	0	Unanimous consensus in favor.
Poll 3: Send [P1206R6] (<code>ranges::to</code>) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	19	9	0	0	0	Unanimous consensus in favor.
Poll 4: Send [P0533R8] (<code>constexpr For <cmath> And <cstdlib></code>) to Library Working	15	13	2	1	0	Consensus in favor.

Group for C++23, classified as an addition ([P0592R4] bucket 3 item).						
Poll 5: Send [P2273R2] (Making <code>unique_ptr constexpr</code>) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	17	16	0	0	0	Unanimous consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. Poll 1: [P2138R4] Rules of Design <=> Wording Engagement

I don't know what the solution is, but this document further solidifies and ingrains our current patterns and processes. Which would be fine, except they are not.

Our current process is an exclusionary set of hurdles for anyone with out the time and money to navigate our gauntlet. This proposal encourages the gauntlet and possibly even adds more hurtles.

Now, there should be hurtles. Particularly since we avoid compatibility breaks, we need to get things right the first time. Decisions are approximately forever.

Also, we do too much to be able to keep track of everything, so pause points that give people time to review and demarcate "this really is moving forward, you better look at it", help those of us whom can't be in multiple rooms at once. (The alternative here is to trust your subgroups)

So I'm conflicted.

— Weakly Against

This paper will have disastrous consequences on WG21 that the committee, its members, and proposal writers are wholly unprepared to deal with. If the idea is that, instead of an open process, we should endeavor to make improving C++ so difficult that it makes COBOL's submission process look easy or similar by comparison, then I suppose this paper is "working as intended". If this paper moves through to plenary and is adopted I can see several people simply not participating in C++ anymore.

— Strongly Against

It seems as a freezing current bad process which detract people and hinders getting new committee members.

— Weakly Against

I'm in favor of strengthening, clarifying, and formalizing the criteria for papers to advance, as I think it will improve quality. However, I worry that this may add complexity to our process, which is already outdated and arcane.

— Neutral

While I think this describes current practice reasonably well, which is helpful, (thus I weakly favor) I'm concerned that the this is might increase the perceived barriers to entry to contributing to WG21.

— Weakly Favor

The problems this process aims to address are plainly real: lots of our limited wording-group bandwidth is consumed by reviewing wording that is unusably incomplete, and disagreements about the design intent of papers approved by design groups have been very problematic. Also, distinguishing papers with approved designs but no wording is simply good process engineering, since they can't be reviewed yet. However, it is not clear that the design groups can reliably evaluate whether wording implements the design intent without duplicating the wording group's work. Also, wordsmiths' time is also limited, and while there's no requirement that they agree with a proposal to review its initial wording, the natural tendency would be to spend their limited time on those that they favor. It doesn't seem right to subject proposals to that sort of pocket veto. A wording review group would of course be well within its rights to suspect that a paper's wording might not be ready and deputize one of its members to ascertain that fact and communicate it to the author (who would be just as capable of securing help with it as they would be under this proposal).

While the paper discusses means of accelerating urgent proposals, it is nonetheless plain that this process would in general extend the review process. That's not intrinsically undesirable, but lengthening the process by fiat (rather than, say, because implementation concerns have been raised) is of questionable value. It's a little unreasonable to require a notional author of one paper to attend every in-person meeting (when those are happening) for years in case one of the many reviews for it occurs at one of them. In particular, having a "park for further design feedback" stage in addition to a "Tentatively Wording" stage seems gratuitous.

— Strongly Against

This proposal will have the following effects if adopted by LEWG:

1. It will cement a complex and baroque process even further into WG21.
2. It will increase the already high gatekeeper discrimination of proposals.
3. It will allow LWG to discriminate against LEWG by discounting their consensus findings.
4. It will place unbounded decision power on individuals with undefined credentials.

Hence it is unconscionable of me to vote anything but strongly against.

— Strongly Against

It's not clear to me that injecting more delay into the process is going to result in more actionable feedback and therefore a better quality standard. It is also quite possible that the stringent gatekeeping would discourage participation and therefore useful feedback.

Additionally, I have not found LEWG wording review to be a worthwhile use of valuable telecon time.

— Weakly Against

I'm not a fan of the requirement that wording must exist to advance out of (L)EWG. My experience suggests that this actively causes harm. What we've seen is that authors now see that they need wording, so they spend time writing wording. The problem is, the wording is often bad anyway -- so you end up with a paper with potentially pages of wording that doesn't really help explain the design because the authors spent time writing bad wording that could have been spent writing clearer motivation, examples, prose, etc. Or the author knows that they can't write wording, so they go try to get wording review for a paper that may just not be a good design, which ends up not being a good use of the wordsmith's time either. Even when the wording is fine, it's just actively distracting from the discussion - which should be at this point about the design. So you get comments about wording issues that are, frankly, completely irrelevant and wholly unproductive. I think the best thing you could say about requiring wording is that it's simply a waste of time in that the author had to spend their time producing wording earlier than necessary. The presented problem (papers show up to CWG/LWG with poor wording) can be solved by just requiring that papers show up to CWG/LWG with good wording - requiring EWG/LEWG to see it seems like a completely needless extra step in the process.

— Weakly Against

I believe this will do a lot more harm than good, and try to solve non-problems.

While C++20 finalization might have been somewhat hectic, our ability to quickly fix issues encountered in 2019 and 2020 is a big part of the quality of the final product. More stiffness is the last thing we need.

This paper, by proposing a constant amount of process regardless of feature size is negatively affecting incremental improvements and small papers.

While many are excited by concepts and pattern matching or executors, we should not discourage papers that propose small incremental improvements like a `contain` method on a string, or making `std::move`nodiscard.

If our process is such that papers can be "too small to be worth our time" there is an issue with our process. And while we should be mindful of our ever-growing backlog, I do not believe making it harder for first-time contributors to participate is a good way to solve this problem.

If it takes 12+ months to add a small uncontroversial function, it simply won't be done and will leave c++ with rough edges. Same for specification cleanups. New contributors will stop showing up.

I trust the chair and our peers to ask of authors an amount of work that is proportional to the perceived risk. I trust the existing process to be good enough (nothing terrible happened in 20, in fact, it is a very successful version). And I trust that being flexible gives the committee the opportunity to improve and learn instead of settings in stone an imperfect process. Enshrining some set of rules gives us less opportunity to improve and adapt to a workload.

I hope we can find ways to make the process more accessible, rather than less. Small group wording reviews, wording guidances, etc would be a better way to reduce the working group workloads. Asking new contributors to find a wordsmith can be intimidating.

Maybe one waiting period would be useful, but having to wait between design and wording, and wording and plenary, while also asking the paper to leave design with wording adds nothing. One period is enough. I do not believe that giving more time to implementers or users to give feedback will stop people from waking up in plenary. We know from experience that more time != more feedback. In a face-to-face meeting context, this might also mean that an author has to travel twice, which might not be possible. Maybe we can advertise the status of papers more clearly so that no one is surprised.

— Strongly Against

It would indeed be useful to recognize the importance of providing wording. I fear we might find our best wordsmiths overflowing with requests, however, which explains why I used "weakly" instead of "strongly" for my vote.

— Weakly Favor

Having been asked about the process for getting a proposal into the standard, I'm happy to have documentation to which I can point. I do have minor concerns about finding wordsmiths, but I feel that can be worked out later if it becomes truly problematic.

— Weakly Favor

We need a checklist and a description for how proposals flow through the committee; it's unreasonable to continue to do that in an ad-hoc manner.

— Strongly Favor

I very much like the process clarification and agree this has been the historical golden standard.

— Strongly Favor

§ 3.2. Poll 2: [\[P2372R1\]](#) Fixing Locale Handling In Chrono Formatters

Better to fix it now than live with it for 30 years.

IMO, almost anything in C++20 is open for change/correction until C++23 is released. Adoption isn't that fast, and any code using new features must be new code, and new code isn't hard to fix.

— Strongly Favor

We want to enable locale-independent formatting as much as possible, while also making explicit locale-dependent formatting possible where needed.

— Strongly Favor

It's unfortunate that we shipped this broken, but we can't reclaim consistency unless we deal with this now, so we should do so.

— Weakly Favor

This fixes an unpleasant inconsistency in `std::format`. I only wish the fix had come two years ago; I dislike continuing to change C++20 after it has shipped, as it adds uncertainty for implementers

— Weakly Favor

Still not enjoying retroactive application [of changes to C++20].

— Weakly Favor

§ 3.3. Poll 3: [\[P1206R6\]](#) `ranges::to`

Converting a range to a concrete container should be easier, and this makes it so.

— Strongly Favor

A long-standing complaint from our developers comes from the verbosity of converting one container to another. The additions proposed here should help alleviate some of those complaints while also making ranges more attractive for newcomers.

— Strongly Favor

I've recently worked on a project that started using `boost::string_view` heavily (C++17 wasn't available). I have become painfully aware of how awkward it is to have views and ranges without convenient conversions between a view and a container. We need something like `ranges::to` in the standard.

— Weakly Favor

The bullet points in the specification of `ranges::to` don't allow for construction using `ranges::begin(r)` and `ranges::end(r)`, even though that is described as one of the options earlier in the document.

— Weakly Favor

The new `from_range_t` design is a lot more extensible, easy to diagnose, and efficient.

— Weakly Favor

§ 3.4. Poll 4: [\[P0533R8\]](#) `constexpr` For `<cmath>` And `<cstdlib>`

It is a significant improvement in applicability of `constexpr` in real-world projects dealing with FP arithmetic.

— Strongly Favor

The functions proposed in the paper, are indeed looking to be "simple", i.e. not more complicated as few operators. Not including them, will lead to user's implementation, that may lead to more subtle bugs.

— Strongly Favor

The worries seemed to be about implementability with respect to `errno`, but that aspect of the problem is now more documented and there seems to be implementation experience on at least one library

— Strongly Favor

Despite the many concerns raised, I still don't think this makes the status quo any worse as regard to constexpr floating point inconsistencies between compile-time and runtime. In fact, this might encourage implementers to guarantee that the maths functions are correctly rounded at compile-time regardless of the target behavior, like GCC does. This will allow many look-up tables to be computed at compile-time instead of relying on code generators.

People will rely on target maths can make sure their operations are not constant-folded. We should encourage more work in this area though!

— Weakly Favor

I like P0533, however, sqrt should also be constexpr. It deserves special treatment because the IEEE standard gives it special treatment: it is the only operation, other than arithmetic operators and std::fma, required to be exact (see IEEE 754-2008, sec 5.4.1). ("Exact" meaning that its error is less than 0.5 ulp)

— Strongly Favor

Not totally convinced this is the best strategy, but I don't have a better suggestion.

— Neutral

The motivation is there, but I'm uncomfortable building more on top of the shaky foundation of constexpr floating point. We need a paper and plenary approval on which things WG21 wants out of constexpr floating point. Do we want compile time and runtime to produce the same results (this would be hard). Do we want the same source to result in the same compile time results across multiple implementations (i.e. do we want compile time calculations portable)? Do we want always-precise results out of compile time calculations? We don't have a uniform answer here, and until we do, I can't be in favor of the proposal.

I'm also not thrilled with the implementation experience on this proposal. Something besides GCCs constant folding implementation really ought to be provided. The existence of builtins isn't sufficient.

— Weakly Against

§ 3.5. Poll 5: [P2273R2] Making `unique_ptr` constexpr

P2273's intro says it all: without P2273, using `new` or `delete` in a constexpr-context would force us back to a pre-c++11 memory-management era.

— Strongly Favor

There seems to be no reason not to do this anymore (I've faced a situation where this would have been useful just last week and was surprised it did not work as I expected it to).

— Strongly Favor

It continues to be a goal to make compile-time programming as similar as possible to runtime programming, to allow as much code as possible to be shared between the two environments. There is no reason to encourage the use of undesirable constructs like manual memory management by making the preferred replacements unavailable during constant evaluation.

— Weakly Favor

This seems a straightforward change enabling some use cases; but the paper itself does appear to lack strong motivation so I only weakly support it.

— Weakly Favor

§ References

§ Informative References

[P0533R8]

Oliver Rosten, Edward Rosten. *constexpr for cmath and cstdlib*. 24 June 2021. URL: <https://wg21.link/p0533r8>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P1206R6]

Corentin Jabot, Eric Niebler, Casey Carter. *Conversions from ranges to containers*. 3 August 2021. URL: <https://wg21.link/p1206r6>

[P2138R4]

Ville Voutilainen. *Rules of Design<=>Specification engagement*. 19 April 2021. URL: <https://wg21.link/p2138r4>

[P2273R2]

Andreas Fertig. *Making std::unique_ptr constexpr*. 5 July 2021. URL: <https://wg21.link/p2273r2>

[P2333R0]

Bryce Adelstein LeBach. *2021 Winter Library Evolution Poll Outcomes*. 12 March 2021. URL: <https://wg21.link/p2333r0>

[P2372R1]

Victor Zverovich, Corentin Jabot. *Fixing locale handling in chrono formatters*. 13 May 2021. URL: <https://wg21.link/p2372r1>